CVM++

Stack-Based Virtual Machine & Custom Compiler

Technical Project Report

Vishwak Motepalli • Srikar Chippada
Indian Institute of Technology Guwahati
2026

ABSTRACT

CVM++ is a complete, from-scratch implementation of a custom scripting language runtime in C++17. The project encompasses the full compilation pipeline — lexical analysis, recursive descent parsing, abstract syntax tree construction, single-pass bytecode compilation, and stack-based virtual machine execution. It demonstrates, in concrete engineering terms, how modern programming language runtimes function internally, without relying on any existing interpreter framework or runtime library.

1. INTRODUCTION

Programming languages are among the most complex software artifacts humans build. Yet their inner workings are rarely taught by building them. CVM++ was conceived as an answer to that gap: a ground-up implementation of every stage of a language runtime, from raw character stream to observable output, in a single cohesive C++ codebase.

The project targets a custom imperative language — also called CVM++ — that supports integer arithmetic, lexically-scoped variables, conditional branching, iterative loops, and interactive I/O. The runtime compiles source programs to a proprietary bytecode instruction set, which is then executed by a purpose-built stack machine.

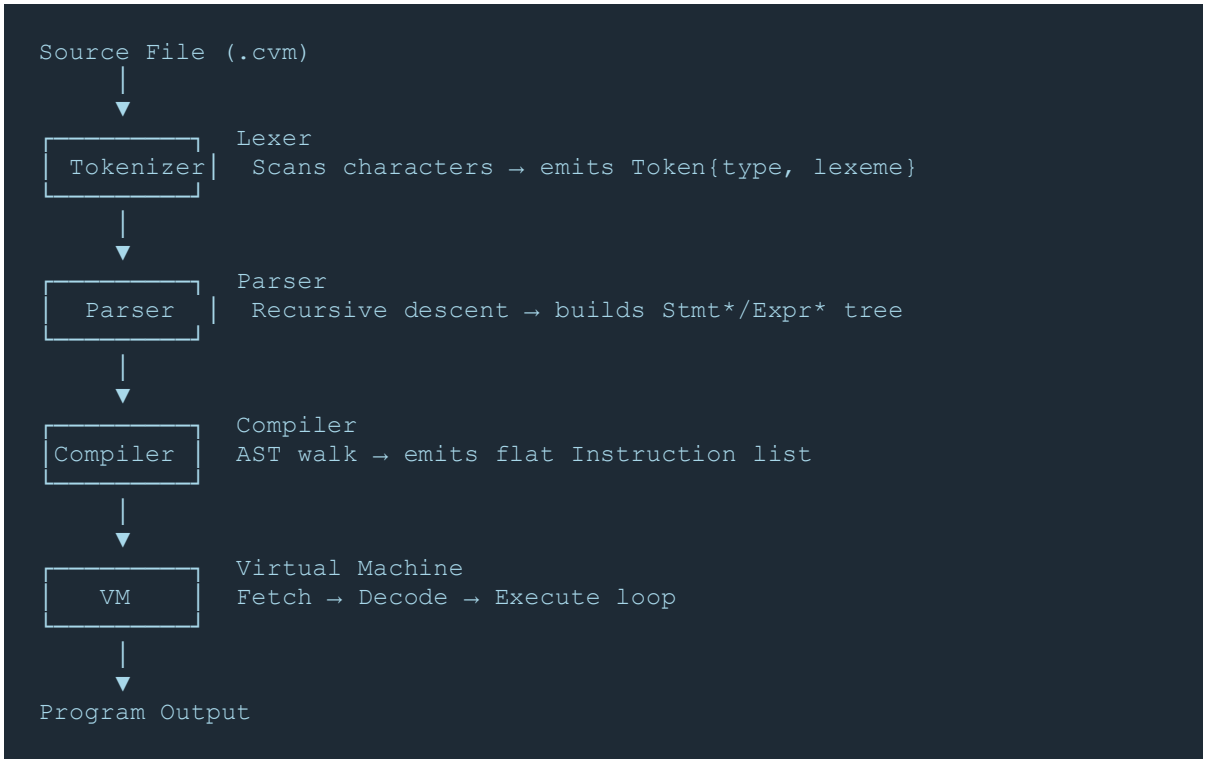
Beyond correctness, the project emphasizes transparency: a debug mode exposes every step of the pipeline — the parsed AST, the emitted bytecode listing, and a step-by-step VM execution trace — making the system useful as a learning and diagnostic tool.

2. SYSTEM ARCHITECTURE

The CVM++ pipeline is strictly layered. Each stage receives a well-defined data structure from the previous stage and produces a well-defined data structure for the next. No stage has knowledge of stages beyond its immediate neighbors.

Stage	Module	Input	Output
1 — Lexer	lexer/	Source string (.cvm)	Token stream
2 — Parser	parser/	Token stream	AST (Program*)
3 — Compiler	compiler/	AST	Instruction vector
4 — VM	vm/	Instruction vector	Execution output

2.1 Pipeline Flow



3. COMPONENT DESIGN

3.1 Lexer

The lexer (lexer/lexer.cpp) implements a single-pass, character-by-character scanner. It maintains a current position cursor and a start marker, producing a Token on each recognition. The lexer handles:

- Integer literals: sequences of decimal digits converted via std::stoi
- Keywords: let, if, else, while, print, input — matched after identifier scanning
- Identifiers: sequences of alphanumeric characters and underscores
- Operators: +, -, *, /, >, <, ==, =, &&, ||
- Delimiters: (,), {, }, ;
- Error reporting: unrecognized characters emit a diagnostic and terminate

3.2 Parser

The parser (parser/parser.cpp) is a hand-written recursive descent parser. It consumes the token stream and produces a heterogeneous tree of Stmt* and Expr* nodes allocated on the heap. The parsing grammar encodes operator precedence directly in the call hierarchy, from lowest to highest:

```
expression
  └─ logical        ( && || )
      └─ equality     ( == )
          └─ comparison  ( > < )
              └─ term       ( + - )
                  └─ factor   ( * / )
                      └─ unary  ( - )
                          └─ primary
```

Statement nodes produced: ExpressionStmt, PrintStmt, IfStmt, WhileStmt, BlockStmt.

Expression nodes produced: LiteralExpr, VariableExpr, BinaryExpr, AssignExpr, InputExpr.

3.3 Abstract Syntax Tree

The AST is defined across ast/expr.h and ast/stmt.h using a simple class hierarchy with virtual destructors. Each node type carries exactly the data it needs. AssignExpr carries an isLet flag to distinguish declarations from reassignments, enabling the compiler to emit OP_DEFINE vs OP_STORE without a separate pass.

The ASTPrinter class (ast/ast_printer.cpp) implements a recursive pretty-printer that emits a Lisp-style S-expression representation of the tree, used by the --debug mode.

3.4 Compiler

The compiler (compiler/compiler.cpp) performs a single-pass post-order walk of the AST, emitting a flat vector of Instruction structs. Each Instruction carries an OpCode, an integer operand, and a string name field.

Control flow is implemented via forward-patched jumps:

- A placeholder JMP_IF_FALSE is emitted with operand 0
- The body statements are compiled
- The true target address is back-patched once known

Lexical scoping is handled by emitting OP_ENTER_SCOPE before each block body and OP_EXIT_SCOPE after. The VM's scope stack mirrors this structure at runtime.

3.5 Virtual Machine

The VM (vm/vm.cpp) is a fetch-decode-execute loop over the instruction vector. It maintains two primary data structures:

- Operand stack (std::stack<int>): holds intermediate values during expression evaluation
- Scope stack (std::vector<std::unordered_map<std::string,int>>): models nested variable environments

Variable lookup (OP_LOAD) walks the scope stack from innermost to outermost, implementing lexical scoping semantics correctly. Variable assignment (OP_STORE) similarly searches outward to find the declaring scope. OP_DEFINE always inserts into the current (innermost) scope.

Division-by-zero is detected at runtime and produces a descriptive error before halting. The --debug flag enables per-instruction tracing showing the stack contents and variable map after each operation.

4. INSTRUCTION SET ARCHITECTURE

The CVM++ ISA is a minimal, stack-oriented instruction set with 20 opcodes. All arithmetic and comparison operations are binary: they pop two operands and push one result. The design prioritises simplicity and inspectability over performance.

Opcode	Operands	Stack Effect	Description
PUSH	n (int)	(— → n)	Push integer constant
POP	—	(a →)	Discard top of stack
ADD	—	(a b → a+b)	Integer addition
SUB	—	(a b → a−b)	Integer subtraction
MUL	—	(a b → a×b)	Integer multiplication
DIV	—	(a b → a÷b)	Integer division, zero-check
AND	—	(a b → a&&b)	Logical AND
OR	—	(a b → a||b)	Logical OR
GT	—	(a b → a>b)	Greater-than, pushes 0 or 1
LT	—	(a b → a<b)	Less-than, pushes 0 or 1
EQ	—	(a b → a==b)	Equality, pushes 0 or 1
DEFINE	name	(a →)	Declare variable in current scope
STORE	name	(a →)	Assign to existing variable
LOAD	name	(— → v)	Push variable value
ENTER_SCOPE	—	(—)	Push new environment frame
EXIT_SCOPE	—	(—)	Pop and discard environment frame
INPUT	—	(— → n)	Read integer from stdin
PRINT	—	(a →)	Pop and write to stdout
JMP	addr	(—)	Unconditional jump to addr
JMP_IF_FALSE	addr	(a →)	Jump to addr if top == 0
HALT	—	(—)	Terminate execution

5. LANGUAGE SPECIFICATION

5.1 Formal Grammar (BNF)

```
program     →  statement* EOF

statement   →  varDecl
            |  assignment
            |  printStmt
            |  ifStmt
            |  whileStmt
            |  block

varDecl     →  'let' IDENTIFIER '=' expression ';'
assignment  →  IDENTIFIER '=' expression ';'
printStmt   →  'print' '(' expression ')' ';'
ifStmt      →  'if' '(' expression ')' block ( 'else' block )?
whileStmt   →  'while' '(' expression ')' block
block       →  '{' statement* '}'

expression  →  logical
logical     →  equality ( ('&&' | '||') equality )*
equality    →  comparison ( '==' comparison )*
comparison  →  term ( ('>' | '<') term )*
term        →  factor ( ('+' | '-') factor )*
factor      →  unary ( ('*' | '/') unary )*
unary       →  '-' unary | primary
primary     →  NUMBER | IDENTIFIER | '(' expression ')' | 'input' '(' ')'
```

5.2 Type System

CVM++ is a dynamically-typed, integer-only language. All values are 32-bit signed integers (C++ int). Boolean results from comparison and logical operators are represented as 1 (true) or 0 (false). There is no implicit conversion and no floating-point support in the current version.

5.3 Scoping Rules

CVM++ implements lexical block scoping. Every block delimited by braces {} introduces a new scope. Variables declared with let in an inner scope shadow variables of the same name in outer scopes. When a block exits (OP_EXIT_SCOPE), all variables declared in that scope are destroyed.

```
let x = 10;
if (x > 0) {
    let x = 99;     // shadows outer x
    print(x);       // prints 99
}
print(x);           // prints 10 (outer x unchanged)
```

6. DEMO PROGRAM — FACTORIAL WITH EXECUTION TRACE

6.1 Source Code

```
let n = input();
let result = 1;

while (n > 1)
{
    result = result * n;
    n = n - 1;
}

print(result);
```

6.2 Generated Bytecode

```
0:  INPUT
1:  DEFINE   n
2:  PUSH     1
3:  DEFINE   result
4:  LOAD     n          ← loop start
5:  PUSH     1
6:  GT
7:  JMP_IF_FALSE  19
8:  ENTER_SCOPE
9:  LOAD     result
10: LOAD     n
11: MUL
12: STORE    result
13: LOAD     n
14: PUSH     1
15: SUB
16: STORE    n
17: EXIT_SCOPE
18: JMP      4          ← back-edge
19: LOAD     result
20: PRINT
21: HALT
```

6.3 Execution Trace (input n = 4)

Step	Instruction	Stack (top → right)	Variables
1	INPUT	[4]	

Step	Instruction	Stack (top → right)	Variables
2	DEFINE n	[ ]	n=4
3	PUSH 1	[1]	n=4
4	DEFINE result	[ ]	n=4, result=1
5	LOAD n	[4]	n=4, result=1
6	PUSH 1	[4, 1]	n=4, result=1
7	GT	[1]	n=4, result=1
8	JMP_IF_FALSE	[ ]	condition TRUE → continue
9	MUL (1×4)	[4]	result→4
10	SUB (4–1)	[3]	n→3
11	GT (3>1)	[1]	continue loop
12	MUL (4×3)	[12]	result→12
13	SUB (3–1)	[2]	n→2
14	GT (2>1)	[1]	continue loop
15	MUL (12×2)	[24]	result→24
16	SUB (2–1)	[1]	n→1
17	GT (1>1)	[0]	FALSE → exit loop
18	JMP_IF_FALSE	[ ]	jump to 19
19	LOAD result	[24]	result=24
20	PRINT	[ ]	OUTPUT: 24
21	HALT	[ ]	done

7. TEST RESULTS

All tests were executed using the interactive Python test runner (run_tests.py). The runner feeds stdin inputs automatically, strips VM output prefixes, and compares normalized output against expected values.

Test Suite	Files	Passed	Status	Notes
tests/valid/	3	3	✓ PASS	Variable decl, assign, nested scopes
tests/expressions/	2	2	✓ PASS	Grouping, operator precedence
tests/controlflow/	3	3	✓ PASS	if/while/nested with live input
tests/invalid/	4	4	✓ PASS	Bad chars, bad if, missing tokens
examples/	7	7	✓ PASS	Factorial, calculator, bool, while…
scripts/	4	4	✓ PASS	Misc scripts, test1 error expected
Σ Total	23	23	✓ PASS	100% pass rate

8. TEAM CONTRIBUTIONS

Module	Owner	Description
Lexer (lexer/)	Vishwak	Character scanner, token stream generation, error diagnostics
Parser (parser/)	Vishwak	Recursive descent parser, grammar enforcement, AST construction
AST (ast/)	Vishwak	Node class hierarchy, AST printer, S-expression debug output
Grammar & Design	Vishwak	Formal grammar specification, language design decisions
REPL Integration	Vishwak	Interactive read-eval-print loop in main.cpp
Compiler (compiler/)	Srikar	AST-to-bytecode compilation, jump back-patching, scope emit
VM Runtime (vm/)	Srikar	Fetch-decode-execute loop, operand stack, scope stack
Opcode ISA Design	Srikar	Instruction set definition, opcode encoding in instruction.h
Scope & Env System	Srikar	Environment stack, variable lookup/define/store semantics
Logical Ops & Control	Srikar	&&, || execution, if/while runtime, JMP back-edge
Debug Trace	Srikar	Per-instruction VM trace output, bytecode printer
Test Suite & Runner	Both	Test cases, run_tests.py interactive runner, coverage

9. FUTURE WORK

The current implementation provides a solid foundation. Planned extensions include:

Enhancement	Priority	Description
Functions & Return	High	User-defined functions with call frames and return values
String Type	High	String literals, concatenation, and print support
Arrays	Medium	Heap-allocated integer arrays with index operators
Floating-Point	Medium	IEEE 754 double support alongside integers
Constant Folding	Medium	Compile-time evaluation of constant sub-expressions
Register-Based VM	Low	Register machine to reduce push/pop overhead
Garbage Collection	Low	Mark-and-sweep GC for heap-allocated objects
JIT Compilation	Low	Native code generation for hot loops via LLVM or dynasm
Optimised VM Dispatch	Low	Computed goto dispatch table to eliminate switch overhead

10. CONCLUSION

CVM++ demonstrates that a complete, working programming language runtime can be built from first principles in a few thousand lines of C++. The project covers every stage of the compilation pipeline with clean separation of concerns: the lexer knows nothing of the parser, the parser knows nothing of the compiler, and the compiler knows nothing of the VM's execution strategy.

The 100% test pass rate across 23 test cases — spanning valid programs, expression evaluation, control flow, and intentional error cases — validates the correctness of the implementation. The debug mode provides a transparent window into every stage of execution, making the system valuable beyond its function as a language runtime.

> *This project gave us concrete, hands-on understanding of how real-world runtimes — the JVM, CPython, the V8 JavaScript engine — work at their core. Building each component from scratch makes the abstractions tangible in a way that reading about them cannot.*